SIDDAGANGA INSTITUTE OF TECHNOLOGY, TUMAKURU- 3
(An Autonomous institution affiliated to Visvesvaraya Technological University- Belagavi, Approved by AICTE,
Accredited by NAAC with 'A++' Grade, Awarded Diamond College Rating by QS I-GAUGE & ISO 9001:2015 certified)

MINI PROJECT REPORT (PHASE-II)

ON

"An Immutable Distributed Data-Sharing Framework Integrating IPFS with Blockchain-Driven Authentication and Attribute-Based Access Control"

submitted in the partial fulfilment of the requirements for V semester,
Bachelor of Engineering in Computer Science and Engineering

Submitted by

AADIL MAJEED	**1SI24CS001**
ANSHUMAN KATOCH	**1SI24CS018**
NIMIT MISHRA	**1SI24CS116**
RAHUL REDDY	**1SI24CS141**

Under the guidance of

Dr. Shruthi K
Assistant Professor
Department of CSE
SIT, Tumakuru-03

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
2026-27

ABSTRACT

Centralised file-sharing systems depend on a single provider to store data, authenticate users and decide who may read a file. This creates a single point of failure, gives users no independent way to verify what happened to their data, and requires complete trust in the operator. Blockchains provide tamper-resistant records but cannot store large files efficiently, while the InterPlanetary File System (IPFS) stores files by content but offers no access control of its own.

This project designs and implements an immutable, distributed data-sharing framework that combines IPFS for encrypted file storage with an Ethereum smart contract for trusted records and access decisions. Users authenticate with a MetaMask wallet by signing a one-time challenge (ECDSA on secp256k1) instead of using passwords. Every file is encrypted with AES-256-GCM before it reaches IPFS, and its SHA-256 hash, content identifier (CID) and access policy are recorded on the blockchain. Access is decided on-chain by attribute-based access control (ABAC) over each user's role, department and organization, and every upload, grant, revocation, request and policy change is kept as a permanent audit event.

Two contributions extend the base design. First, **time-limited delegation of access management** lets an owner hand a trusted user, such as a teaching assistant, the right to grant and revoke access to a file for a fixed period, under least-privilege rules enforced by the contract. Second, **revocation-aware key rotation with file versioning** re-encrypts a file under a fresh key on revocation or key compromise, destroys the old key (crypto-shredding) and anchors every version on-chain.

The system was built with Solidity, Hardhat, Node.js/Express and React, and verified with 44 automated contract tests and end-to-end checks. Measurements on a local Ethereum network show that uploading a file record costs about 507k gas, access decisions are free view calls, and a complete key rotation of a 25 MB file finishes in about 0.54 s. The result is a working proof of concept of secure, transparent and controllable decentralised file sharing.

Keywords: *IPFS, blockchain, smart contract, attribute-based access control, wallet authentication, AES-256-GCM, delegation, key rotation, crypto-shredding.*

Table of Contents

Chapter 1: Introduction 1
1.1 Overview of the Project 1
1.2 Background and Earlier Work 2
1.3 Motivation 2
1.4 Organization of the Report 3
Chapter 2: Literature Survey 4
IPFS-Based File Storage, Access Control and Authentication 4
Decentralized Privacy-Preserving Data Sharing for Electronic Health Records 4
Railway Network Security with Blockchain and Attribute-Based Encryption 4
Blockchain–IPFS Permissioned Architecture for Healthcare Data Governance 5
Blockchain Framework for Secure Data Sharing Using IPFS and ECC 5
File Sharing System with Integrity Assurance and Audit Trail 6
Trusted Storage and Dynamic Access Control of Smart Grid Data 6
Privacy-Preserving Access Control with Optimized User and Attribute Revocation 7
Securing IPFS with a Parallelized AES-256-GCM Engine 7
2.1 Limitations of Existing Work 8
Chapter 3: Problem Statement and Objectives 10
3.1 Problem Statement 10
3.2 Objectives 10
3.3 Novelty of the Work 11
Chapter 4: Requirements Specification 12
4.1 Functional Requirements 12
4.2 Non-Functional Requirements 12
4.3 Software and Hardware Requirements 13
Chapter 5: System Design & Methodology 15
5.1 System Architecture 15
5.2 Data Flow Diagrams 16
5.2.1 Wallet Login Sequence 17
5.2.2 File Upload Sequence 18
5.3 Algorithms 19
5.3.1 Attribute-Based Access Decision 19
5.3.2 Wallet Authentication 21
5.3.3 Encrypted Upload and Verified Download 21
5.3.4 Time-Limited Delegation (Novelty 1) 21
5.3.5 Revocation-Aware Key Rotation (Novelty 2) 22

5.4 Data Design 24
Chapter 6: Implementation Details 26
6.1 Programming Languages Used 26
6.2 Technologies and Tools Used 26
6.2.1 Smart Contract Interface 27
6.2.2 Backend API 27
6.2.3 User Interface 28
6.3 Dataset Details 28
Chapter 7: Results 29
7.1 Screenshots 29
7.2 Functional Testing 31
7.3 Analysis 32
7.3.1 Gas Cost of Contract Operations 32
7.3.2 Cryptographic Processing Time 33
7.3.3 End-to-End Latency 34
7.3.4 Comparison with Existing Work 36
Chapter 8: Conclusion 38
8.1 Limitations 38
8.2 Future Work 38
References 40

List of Figures

Figure 5.1: Three-layer system architecture 15
Figure 5.2: Data flow diagram – Level 0 (context) 16
Figure 5.3: Data flow diagram – Level 1 17
Figure 5.4: Sequence diagram – wallet login 18
Figure 5.5: Sequence diagram – encrypted upload 19
Figure 5.6: Flowchart of the on-chain access decision 20
Figure 5.7: Lifetime of a delegation and the grants made under it 22
Figure 5.8: Sequence diagram – three-phase key rotation 23
Figure 5.9: On-chain data model (entity–relationship view) 24
Figure 7.1: Dashboard of the administrator, with identity and system status 29
Figure 7.2: Upload page with the access policy and two-step progress 29
Figure 7.3: Shared With Me page for a CSE student, with request by ID 30
Figure 7.4: Dashboard of a registered student with the MetaMask wallet open 30
Figure 7.5: Audit log read directly from smart-contract events 30
Figure 7.6: Gas used per operation (novelty functions in blue) 33
Figure 7.7: Cryptographic processing time versus file size (log–log) 34
Figure 7.8: End-to-end latency by file size 35

List of Tables

Table 2.1: Summary of the surveyed works 7
Table 4.1: Functional requirements 12
Table 4.2: Non-functional requirements 12
Table 4.3: Software requirements 13
Table 4.4: Hardware requirements 14
Table 5.1: Mapping of entities to contract storage 24
Table 6.1: Technologies and their role in the project 26
Table 6.2: Main FileStorage contract functions 27
Table 6.3: Backend REST endpoints 27
Table 7.1: Automated smart-contract tests 31
Table 7.2: End-to-end test scenarios 31
Table 7.3: Gas used by each contract transaction 32
Table 7.4: Encryption, decryption and hashing time 33
Table 7.5: End-to-end operation time 34
Table 7.6: Feature comparison with the surveyed works 36

Chapter 1: Introduction

The volume of digital information that institutions and individuals exchange keeps growing, and with it the need for storage and sharing methods that are secure, reliable and verifiable. Most file-sharing services today are centralised: one organisation stores the data, manages the user accounts and decides who can read what. This is convenient, but it concentrates risk. A server failure or breach affects every user, an insider can change permissions without trace, and users must simply trust that the provider behaves correctly.

1.1 Overview of the Project

This project builds a decentralised file-sharing framework in which no single component has to be trusted for everything. Responsibilities are split across technologies that are each strong at one job:

- **IPFS** stores the files. Each file is addressed by a content identifier (CID) derived from its bytes, so any change to the content produces a different CID.
- **An Ethereum smart contract** (`FileStorage`) stores what must be trustworthy: who the users are and their attributes, each file's CID, SHA-256 hash and access policy, explicit grants and revocations, delegations, version history and an append-only audit log of events.
- **Wallet authentication** with MetaMask replaces usernames and passwords. A user proves ownership of a blockchain address by signing a one-time message.
- **Attribute-based access control (ABAC)** decides access by comparing a user's role, department and organization with the policy attached to each file. The decision runs inside the smart contract, so it is transparent and cannot be secretly altered.
- **Encryption** with AES-256-GCM ensures that the content stored on IPFS is unreadable without a key, because IPFS itself does not restrict who can fetch a CID.

A React web application gives users pages to upload files, manage access, view files shared with them, see an audit log and, for the administrator, register users. A Node.js backend performs encryption, talks to IPFS and releases a file only after the contract confirms access. On top of this base, the project adds two novel features: time-limited delegation of access management and revocation-aware key rotation with on-chain file versioning.

1.2 Background and Earlier Work

The project builds on a line of foundational work. Nakamoto's Bitcoin paper (2008) introduced the blockchain as a replicated, tamper-evident ledger maintained without a central authority. Wood's Ethereum Yellow Paper (2014) extended this idea to smart contracts, programs whose execution every node verifies, which makes on-chain access rules possible. Benet (2014) proposed the InterPlanetary File System (IPFS), a peer-to-peer file system in which content is addressed by a hash of its bytes rather than by its location.

Access control based on attributes was formalised by Hu et al. in NIST Special Publication 800-162 (2014), and Bethencourt, Sahai and Waters (2007) introduced ciphertext-policy attribute-based encryption (CP-ABE), in which data is encrypted under a policy that only matching attribute keys can satisfy. Combining these ideas, Steichen et al. (2018) proposed blockchain-based, decentralised access control for IPFS and showed why large files should stay off-chain. Hussien et al. (2021) designed blockchain-based access control for personal health records stored on IPFS using attribute-based searchable encryption. Zhao et al. (2022) built an attribute-based access-control scheme for data sharing on Hyperledger Fabric with encrypted data on IPFS, and Huang, Chang and Wu (2022) presented an IPFS and blockchain file-sharing system with group key management through an IPFS proxy.

The cryptographic primitives used in this project are standardised: AES in Galois/Counter Mode (NIST SP 800-38D, 2007), the SHA-256 hash function (FIPS 180-4, 2015), Ethereum's signed-message format (EIP-191, 2016), JSON Web Tokens (IETF RFC 7519, 2015) and the key-management guidance of NIST SP 800-57 (2020). These earlier works and standards form the background of the project; Chapter 2 surveys recent research published from 2023 onwards.

1.3 Motivation

Several practical problems motivate this work:

- **Single point of failure and trust.** Centralised servers can fail, be breached, or have records changed by an insider. Users cannot independently verify who accessed their files.
- **Blockchains are poor file stores.** Every node replicates the chain, so storing large files directly on-chain is slow and costly. Recent systems therefore keep data off-chain, on IPFS or encrypted storage, and anchor only hashes and metadata on the blockchain [1]–[4].

- **IPFS has no built-in access control.** Anyone who knows a CID can retrieve the content from the network, so confidential data needs encryption and a separate authorisation layer.
- **Fine-grained, flexible sharing.** Academic and organisational data is naturally shared by attributes such as "all CSE faculty", not by long lists of names, which makes ABAC a better fit than identity lists.
- **Real workflows need delegation and safe revocation.** Faculty often rely on assistants to manage access temporarily, and revoking access or recovering from a leaked key should not require rebuilding the whole system.

Existing research solutions often address these needs with heavy cryptography such as CP-ABE, trusted hardware or permissioned blockchain platforms [2], [3], [4], which are hard to reproduce in a student project. This motivates a simpler, working design that still demonstrates the key properties end to end.

1.4 Organization of the Report

The rest of this report is organised as follows. **Chapter 2** surveys recent work (2023 onwards) on blockchain and IPFS-based secure data sharing and identifies its limitations. **Chapter 3** states the problem, the objectives and the novelty. **Chapter 4** lists the functional, non-functional, software and hardware requirements. **Chapter 5** presents the system architecture, data flow diagrams, algorithms and data design. **Chapter 6** describes the implementation, languages and tools. **Chapter 7** shows screenshots, test results and performance analysis with comparison tables and plots. **Chapter 8** concludes the report and outlines future work, followed by the references.

Chapter 2: Literature Survey

This chapter reviews nine recent research works, published from 2023 onwards, that combines blockchain, off-chain storage such as IPFS, encryption and access control for secure data sharing. Earlier foundational work is summarised in Section 1.2. For each work, the approach, the reported results and its relevance to this project are noted.

IPFS-Based File Storage, Access Control and Authentication

Antony Saviour and Samiappan [1] proposed an IPFS-based blockchain model for secure data transfer that integrates authentication and access control. The model involves a smart contract, a data owner, a data requester, the blockchain and IPFS, and its workflow covers user registration, storage, encryption, authentication, access control and decryption.

Relevance: this work shows that authentication, access control, a blockchain and IPFS can be combined into a single data-sharing workflow. Our project follows a simplified version of this end-to-end workflow, with wallet signatures for authentication and attribute-based rules for access control.

Decentralized Privacy-Preserving Data Sharing for Electronic Health Records

Rajakarthik, Sumalatha and Rajalingam [2] proposed blockEHR, a decentralised system for sharing electronic health records. Data from Internet of Medical Things (IoMT) devices is verified at the edge using digital signatures, identity cross-referencing and GPS-based location checks, and then shared over a permissioned private Ethereum network (Hyperledger Besu). Smart contracts with role-based access control protect the data, AES-256 encryption keeps records confidential and SHA-256 hashing ensures integrity, while encrypted records are kept off-chain and access requests, approvals and revocations are logged immutably on-chain. The authors report a throughput of about 1,850 records per second with low latency.

Relevance: the work confirms the combination used in our project, symmetric AES-256 encryption with SHA-256 integrity hashes, smart-contract access control and an on-chain audit trail, and shows that it performs well in practice. Its access control is role-based and its network is permissioned; our project uses attributes (role, department and organization) and a public-style Ethereum network.

Railway Network Security with Blockchain and Attribute-Based Encryption

Ahmed et al. [3] presented a blockchain-based framework for secure data sharing in railway networks. Data is encrypted with ciphertext-policy attribute-based encryption (CP-ABE) and

stored on IPFS; the storage address is encrypted with AES and recorded on an Ethereum proof-of-authority blockchain, while an Intel SGX trusted execution environment protects sensitive processing. Smart contracts handle data sharing and access control. Compared with earlier schemes, the authors report a 27% reduction in computational cost, a 4% decrease in authentication time, an 8% improvement in peer-to-peer connection set-up and a 23% decline in data-sharing cost, with a peak throughput of 450 transactions per second. They also note that policy revocation shows higher latency because it requires CP-ABE key generation.

Relevance: this work demonstrates attribute-based control together with blockchain and IPFS, and its observation that revocation is costly under CP-ABE directly motivates our lighter approach: attribute rules are checked in the smart contract, and revocation is strengthened by re-encrypting a single file under a fresh symmetric key (Novelty 2) instead of regenerating attribute keys. It also depends on specialised SGX hardware, which our design does not need.

Blockchain–IPFS Permissioned Architecture for Healthcare Data Governance

Shaheen et al. [4] proposed a permissioned architecture that combines Hyperledger Fabric with IPFS for large medical datasets from Medical Internet of Things (MIoT) environments. Encrypted clinical records are stored off-chain on IPFS and only their cryptographic hashes, timestamps and access permissions are anchored on the blockchain, which enables immutable integrity checks. Smart contracts enforce fine-grained, identity- and role-based access control and keep automatic audit logs, and an encrypted querying mechanism lets authorised clinicians retrieve patient histories. Using Hyperledger Caliper, the authors report 145–162 transactions per second, about 38% lower data-retrieval latency and about 72% less on-chain storage than blockchain-only storage.

Relevance: this work strongly supports the core design choice of this project, keeping encrypted files on IPFS and anchoring only hashes and permissions on-chain, and quantifies its storage benefit. Our project applies the same pattern to general-purpose file sharing on Ethereum, with attribute-based rather than role-based access.

Blockchain Framework for Secure Data Sharing Using IPFS and ECC

Ghanmi et al. [5] proposed a decentralised architecture for secure data sharing and access control in untrusted cloud environments, implemented on Ethereum with Hardhat and MetaMask. Files are encrypted locally with AES before being uploaded to IPFS, and their metadata is recorded on the blockchain through smart contracts. Users are authenticated by binding a unique identifier to their public key in a smart contract, and access is managed

through contracts that store the list of authorised users (black-list check, identity, access, grant and revocation contracts). An ECDSA- and timestamp-based verification mechanism prevents replay attacks and detects unauthorised modification, and elliptic-curve cryptography (ECC) protects key management. The authors report lower computation and communication costs than related architectures, but state that their approach does not yet support user revocation or access-policy updates.

Relevance: this is the work closest to ours in technology: Ethereum smart contracts, MetaMask, AES encryption before IPFS and signature-based protection against replay. Our project adds what the authors list as future work, user revocation and policy updates, and replaces lists of authorised users with attribute-based rules.

File Sharing System with Integrity Assurance and Audit Trail

Hidayah and Ibrahim [6] designed and implemented a file-sharing system for a regional library and archives department in West Java using the Design Science Research Methodology. The system uses the permissioned Hyperledger Besu blockchain with IPFS: documents are encrypted with AES-256 and stored off-chain on IPFS, metadata and an immutable audit trail are kept on-chain, and smart contracts permanently record changes to access rights under role-based access control. Integrity is verified by comparing SHA-256 hashes. In testing, document manipulation was detected in all scenarios, the revocation mechanism blocked users whose rights had been removed, blockchain transactions took about 4.5 s on average and IPFS uploads about 248 ms.

Relevance: this system has almost the same goals as ours (encrypted IPFS storage, SHA-256 integrity checks, revocation and an on-chain audit trail for file sharing) and confirms that the design is practical for real institutions. Our project differs by using attribute-based instead of role-based access and by adding delegation, key rotation and version history.

Trusted Storage and Dynamic Access Control of Smart Grid Data

Li et al. [7] proposed a dynamic access-control scheme for smart-grid data that integrates the Hyperledger Fabric consortium blockchain, IPFS and CP-ABE. IPFS stores the large volumes of grid monitoring data while the blockchain keeps only data hashes and access-policy indices, reducing on-chain storage, and CP-ABE provides fine-grained control so that only users whose attributes satisfy a policy can decrypt the data. The authors report that CP-ABE encryption is about 40% more efficient than traditional RSA.

Relevance: the work applies the same IPFS-plus-blockchain, attribute-based pattern to a different domain, energy systems, which shows the generality of the approach. Like [3], it relies on CP-ABE and a permissioned platform, which our design avoids.

Privacy-Preserving Access Control with Optimized User and Attribute Revocation

Worapaluk and Fugkeaw [8] addressed revocation, a known weakness of CP-ABE, for sharing electronic health records in the cloud. Their scheme, R-CP-ABE-ORP, combines ciphertext aggregation, lazy re-encryption with revocation tokens, proxy-assisted lightweight re-encryption, a blockchain for revocation records and Bloom filters for fast queries of affected ciphertexts. It supports both user and attribute revocation with forward and backward security, and the authors report better revocation efficiency, computational cost and query performance than related schemes.

Relevance: the amount of machinery needed to make revocation efficient under CP-ABE (re-encryption, proxies and query structures) supports our decision to handle revocation differently: an on-chain revocation flag takes effect immediately, and the optional key rotation re-encrypts only the affected file under a fresh symmetric key and destroys the old one.

Securing IPFS with a Parallelized AES-256-GCM Engine

El Sehayl et al. [9] observed that IPFS lacks security measures such as encryption to keep stored data confidential, and proposed a parallel encryption engine incorporated into IPFS. File chunks are encrypted with AES-256-GCM before they are distributed across the network, using OpenMP parallelism and AES-NI hardware instructions. Their evaluation of encryption speed, storage overhead and retrieval efficiency shows that the engine outperforms OpenSSL and a serial implementation and that the trade-off between security and performance is minimal.

Relevance: this work independently confirms two choices made in our project: encrypting data with AES-256-GCM before it reaches IPFS, and the finding that such encryption adds little overhead (Section 7.3.2).

Table 2.1: Summary of the surveyed works

Work	Platform / storage	Access control	Reported results	Idea used in this project
Antony Saviour & Samiappan [1], 2023	Blockchain + IPFS	Authentication + access control via smart contract	End-to-end secure transfer workflow	Single workflow from registration to decryption
Rajakarthik et al. [2], 2026	Private Ethereum (Besu) + off-chain store	Role-based, smart contracts	≈1,850 records/s	AES-256 + SHA-256, on-chain audit of access

Work	Platform / storage	Access control	Reported results	Idea used in this project
Ahmed et al. [3], 2026	Ethereum PoA + IPFS + Intel SGX	CP-ABE	−27% compute cost, 450 TPS peak	Attribute-based control; motivates lighter revocation
Shaheen et al. [4], 2026	Hyperledger Fabric + IPFS	Identity/role-based, smart contracts	145–162 TPS, −72% on-chain storage	Files on IPFS, only hashes on-chain
Ghanmi et al. [5], 2025	Ethereum + IPFS, ECC	Authorised-user lists in smart contracts	Lower computation and communication cost	MetaMask/Ethereum stack; anti-replay signatures
Hidayah & Ibrahim [6], 2026	Hyperledger Besu + IPFS	Role-based, smart contracts	All tampering detected; tx ≈4.5 s	Integrity check and audit trail for file sharing
Li et al. [7], 2025	Hyperledger Fabric + IPFS	CP-ABE	≈40% faster encryption than RSA	Only hashes and policy indices on-chain
Worapaluk & Fugkeaw [8], 2026	Cloud + blockchain	Revocable CP-ABE	Faster user/attribute revocation	Motivates lightweight revocation and re-keying
El Sehayl et al. [9], 2026	IPFS	Encryption only	Beats OpenSSL; minimal overhead	AES-256-GCM before IPFS

2.1 Limitations of Existing Work

The surveyed works show that blockchain and IPFS can provide secure, auditable data sharing, but several limitations remain, especially for a practical, reproducible implementation:

1. **Complex cryptography and special hardware.** Attribute-based designs rely on CP-ABE [3], [7], [8], and [3] also requires Intel SGX trusted hardware; these are difficult to implement, verify and deploy in a small prototype.
2. **Costly or missing revocation.** Under CP-ABE, revocation shows higher latency because new keys must be generated [3], and making it efficient needs ciphertext aggregation, lazy and proxy re-encryption and Bloom filters [8]. The Ethereum-based framework of [5] does not support user revocation or policy updates at all. None of the works describes recovering from a leaked encryption key.
3. **Role-, identity- or list-based control.** Several systems grant access by role [2], [4], [6] or keep lists of authorised users [5], which is less flexible than combining several attributes such as role, department and organization.

4. **Permissioned infrastructure.** Hyperledger Fabric [4], [7], Hyperledger Besu [2], [6] and proof-of-authority networks [3] need dedicated set-up and membership management, which raises the barrier to demonstration and testing.
5. **No delegation of management.** In all nine works, access is granted and revoked by the data owner or the system's authorities; none provides a way to let an assistant manage access to a file for a limited time.
6. **No verifiable version history.** The surveyed works anchor the hash of the current data but do not describe an on-chain history of file versions, so updates or re-encryptions cannot be traced.
7. **Domain-specific designs.** Most works target one domain: healthcare [2], [4], [8], railways [3], smart grids [7] or government archives [6], which limits their direct reuse for general file sharing.
8. **IPFS alone does not protect data.** IPFS provides neither encryption nor access control [9]; possession of a CID is enough to fetch content, so every design must add encryption and an authorisation layer on top of IPFS.

This project addresses limitations 1, 3, 4, 7 and 8 with a simpler, general-purpose architecture that enforces attribute-based rules in an Ethereum smart contract over AES-encrypted IPFS storage, and limitations 2, 5 and 6 with the two novel features described in Chapter 3.

Chapter 3: Problem Statement and Objectives

3.1 Problem Statement

Centralised file-sharing systems depend on central servers and authorities to store data, authenticate users and manage permissions. They introduce a single point of failure and make it difficult to provide transparent, tamper-resistant records of data-related activity. Blockchains provide decentralisation and immutability but are not suited to storing large files, while IPFS stores files efficiently but does not control who can read them. Advanced research solutions close this gap with complex cryptography that is hard to deploy, and they rarely support delegated management or recovery from leaked keys.

The problem addressed by this project is: to design and develop a distributed data-sharing framework that uses IPFS for encrypted file storage, a blockchain smart contract for trusted, tamper-resistant records, wallet signatures for user authentication, and attribute-based access control for authorisation, while also supporting time-limited delegation of access management and revocation-aware key rotation with verifiable versioning. The framework must balance security, decentralisation, integrity, auditability and implementation simplicity.

3.2 Objectives

1. To develop a secure and decentralised data-sharing framework that integrates blockchain and IPFS, with IPFS storing encrypted files and the blockchain maintaining the important file records.
2. To implement user authentication and identity verification with blockchain wallets and cryptographic signatures, so that only authenticated users can interact with the system.
3. To implement attribute-based access control (ABAC) in smart contracts, so that access to a file depends on attributes such as role, department and organization.
4. To provide secure file management with access control and revocation, letting authorised users retrieve shared files and letting owners grant, modify or revoke permissions.
5. To ensure data integrity, tamper resistance and auditability using IPFS content identifiers, SHA-256 hashes and blockchain event records.
6. To provide time-limited, least-privilege delegation of access management, so that an owner can let a trusted user manage access to a file for a fixed period.

7. To provide revocation-aware key rotation with crypto-shredding and on-chain file versioning, so that a file can be re-keyed after revocation or key compromise and its full history remains verifiable.

3.3 Novelty of the Work

Objectives 1–5 follow the established architecture from the literature in a lightweight form. Objectives 6 and 7 are the novel contributions of this project:

- **Novelty 1 – Time-limited delegation of access management.** An owner can delegate the right to grant and revoke access to one file to a registered user until a chosen time. A delegate cannot change the policy, deactivate the file, delegate further, grant itself access or undo a revocation made by the owner, and every grant it makes ends automatically when the delegation ends or is removed. In the works surveyed, only the owner or a central authority manages access.
- **Novelty 2 – Revocation-aware key rotation and file versioning.** On revocation or suspected key compromise, the file is re-encrypted under a fresh key and stored as a new IPFS version; once the blockchain records the rotation, the old key is destroyed and the old copy unpinned, so old encrypted copies can never be decrypted again. Every version (initial upload, rotation after revoke, rotation after compromise, new content) is recorded on-chain with its CID, hash, author and time. This offers practical revocation and compromise recovery without the complexity of CP-ABE re-keying.

Chapter 4: Requirements Specification

This chapter specifies what the system must do (functional requirements), the qualities it must have (non-functional requirements) and the software and hardware used to build and run it.

4.1 Functional Requirements

Table 4.1: Functional requirements

ID	Requirement	Actor
FR1	Connect a MetaMask wallet and log in by signing a one-time message; receive a session token (JWT).	All users
FR2	Register a wallet with role, department and organization; look up a user's attributes.	Admin
FR3	Upload a file: encrypt it, store it on IPFS, and record its CID, SHA-256 hash and access policy on-chain.	Registered user
FR4	Define and update a file's access policy (role, department, organization; empty = any).	Owner
FR5	Decide access on-chain by the ABAC rules and explain the reason for each decision.	System
FR6	Download a file only if access is allowed; decrypt it and verify its hash against the blockchain.	Authorised user
FR7	Grant access to a specific wallet, permanently or until a chosen time; revoke access.	Owner, delegate
FR8	Deactivate or reactivate a file.	Owner
FR9	Delegate management of a file to a registered user until a chosen time; remove a delegate.	Owner
FR10	Rotate a file's encryption key after a revocation or key compromise, destroying the old key.	Owner, delegate*
FR11	Upload a new content version; view the version history; download older versions whose key still exists.	Owner
FR12	Request a file by ID with the attempt recorded on-chain; view the full audit log.	All users

* A delegate may rotate the key only together with a revocation.

4.2 Non-Functional Requirements

Table 4.2: Non-functional requirements

Quality	Requirement and how it is met
Security	Files are encrypted with AES-256-GCM before storage; keys are stored wrapped with a master key; no passwords are stored; login nonces are single-use and expire in 5 minutes.

Quality	Requirement and how it is met
Integrity	The SHA-256 hash of every file is stored on-chain and re-checked on every download; GCM authentication tags detect modified ciphertext.
Auditability	Every state change emits a smart-contract event that cannot be edited or deleted.
Transparency	Access decisions are computed by public contract code (`canAccess`), not hidden server logic.
Availability	Files are stored on IPFS (content-addressed, replicable); the system falls back to a local store if no IPFS node is running.
Performance	Upload, download and access checks complete in well under one second for files up to 25 MB on a laptop (Chapter 7).
Usability	A web interface with clear pages, step indicators and human-readable reasons for allow/deny decisions.
Maintainability	Separate layers (contract, backend services, React components); 44 automated tests.
Portability	Runs on Windows, Linux or macOS with Node.js; one command (`npm start`) starts all parts.

4.3 Software and Hardware Requirements

Table 4.3: Software requirements

Software	Version used	Purpose
Operating system	Windows 11 (any OS with Node.js)	Development and execution
Node.js / npm	18.20.8 / 10.8.2	Runtime for backend, tooling and build
Solidity compiler	0.8.24 (via-IR, optimizer on)	Smart contract language
Hardhat + Toolbox	2.29.1 / 5.0.0	Compile, test, local Ethereum network
ethers.js	6.17.0	Blockchain access from backend and frontend
Express / jsonwebtoken / multer	4.22.3 / 9.0.3 / 1.4.5	REST API, JWT sessions, file uploads
React / Vite	18.3.1 / 5.4.21	Web user interface and dev server
MetaMask	Browser extension (current)	Wallet: keys, signing, transactions
IPFS Kubo (optional)	Current release	IPFS node for content-addressed storage
Browser	Google Chrome / Microsoft Edge	Running the web application
Git / GitHub	Current	Version control and code hosting

Table 4.4: Hardware requirements

Component	Minimum	Used for development
Processor	Dual-core 64-bit, 2 GHz	12th Gen Intel Core i5-1235U
Memory (RAM)	4 GB	8 GB
Storage	2 GB free (dependencies + files)	SSD
Network	Not required for the local network; internet for public testnet / IPFS network	Wi-Fi

Chapter 5: System Design & Methodology

The system follows a three-layer architecture. The browser holds the user interface and the wallet; a Node.js backend performs encryption and file handling; and the trust and storage layer consists of the `FileStorage` smart contract and IPFS. This chapter describes the architecture, data flows, algorithms and on-chain data design.

5.1 System Architecture

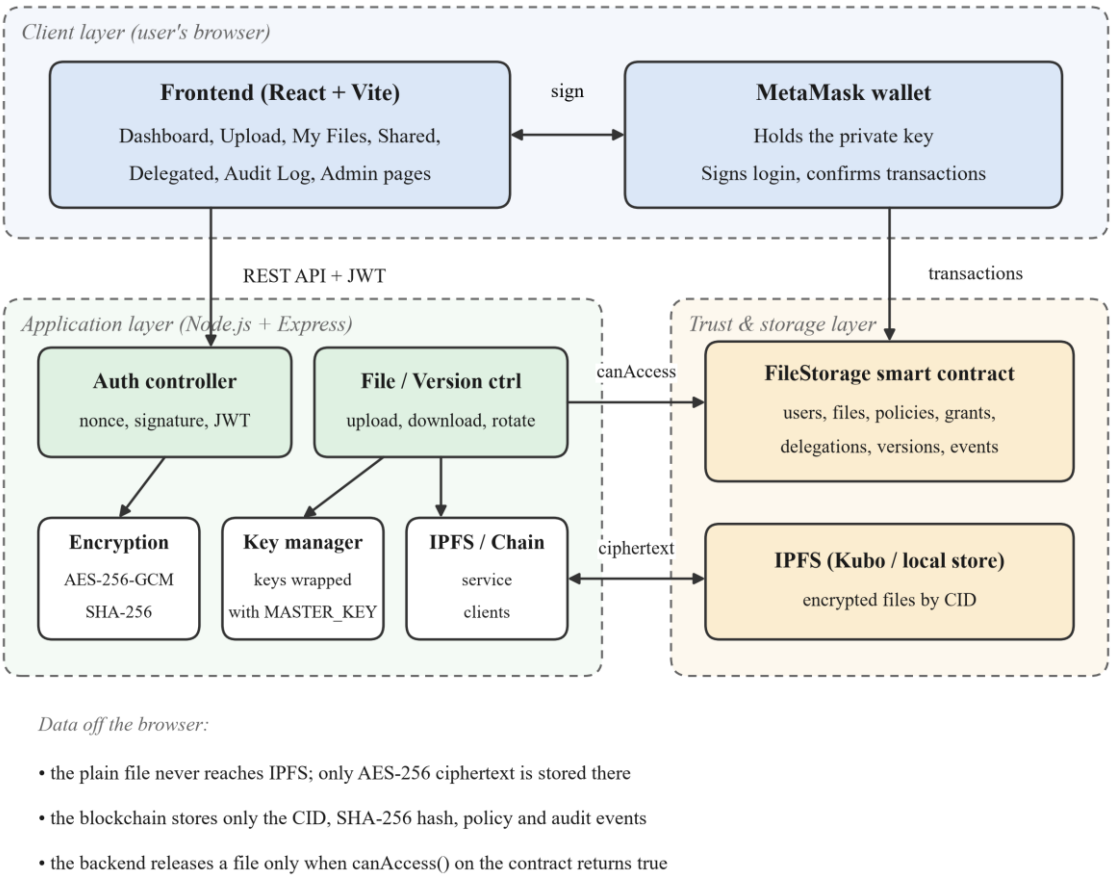


Figure 5.1: Three-layer system architecture

Figure 5.1 shows the components and the data that passes between them:

- **Client layer.** The React frontend provides the pages. MetaMask holds the user's private key, signs the login challenge and confirms every transaction, so the application never sees the key.
- **Application layer.** The Express backend verifies login signatures and issues JWTs, encrypts and decrypts files, stores each file key wrapped with a server master key, and

talks to IPFS. It reads the contract to enforce access but never sends transactions on a user's behalf.

- **Trust and storage layer.** The `FileStorage` contract stores users, files, policies, grants, delegations and versions, makes every access decision and emits audit events. IPFS stores only ciphertext.

Writes that change trusted state (registering users, uploading file records, granting, revoking, delegating, rotating keys) are always signed by the user in MetaMask and sent directly to the contract. The backend is trusted only to release a decrypted file when `canAccess` returns true, and its key store holds only wrapped keys.

5.2 Data Flow Diagrams

The context diagram (Figure 5.2) shows the system as one process with its external entities. The administrator supplies user attributes; file owners supply files, policies and management actions and receive CIDs, versions and audit records; requesters authenticate and request files; and delegates grant or revoke access within their time limit.

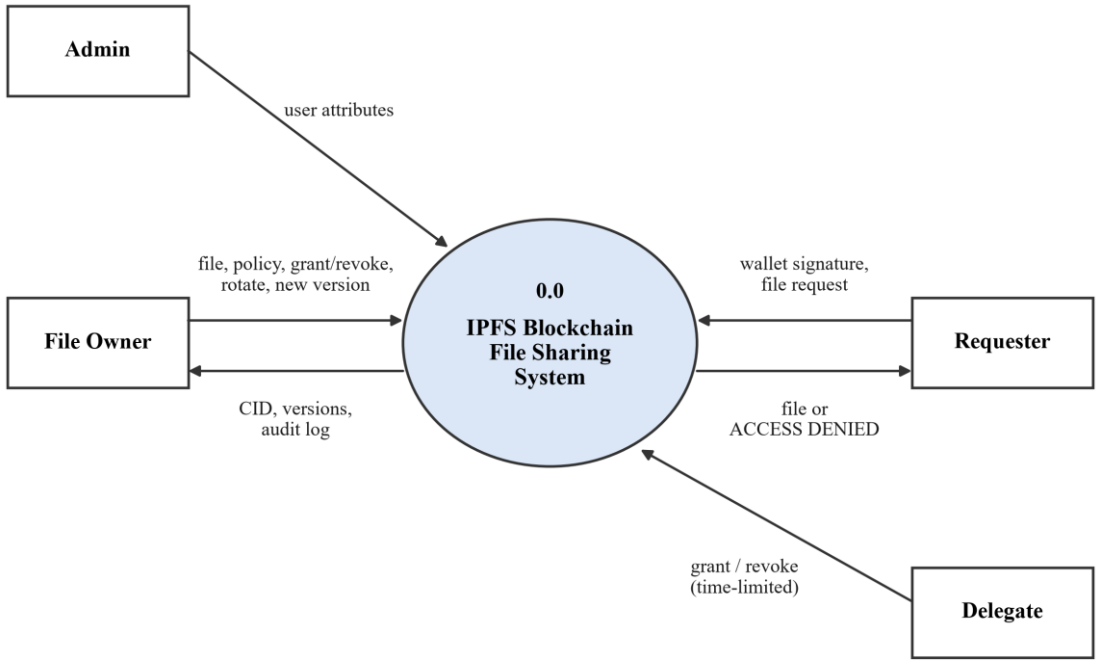


Figure 5.2: Data flow diagram – Level 0 (context)

The Level 1 diagram (Figure 5.3) breaks the system into six processes and three data stores. Process 1.0 authenticates wallets and issues JWTs. Process 2.0 registers attributes in the blockchain ledger (D1). Process 3.0 encrypts files, stores ciphertext in IPFS (D2) and keys in the key store (D3), and records the CID, hash, policy and version in D1; key rotation also runs

here. Process 6.0 records grants and delegations in D1. Process 4.0 reads policies, attributes and grants from D1 to decide access, and process 5.0 downloads, decrypts and verifies the file only when 4.0 allows it.

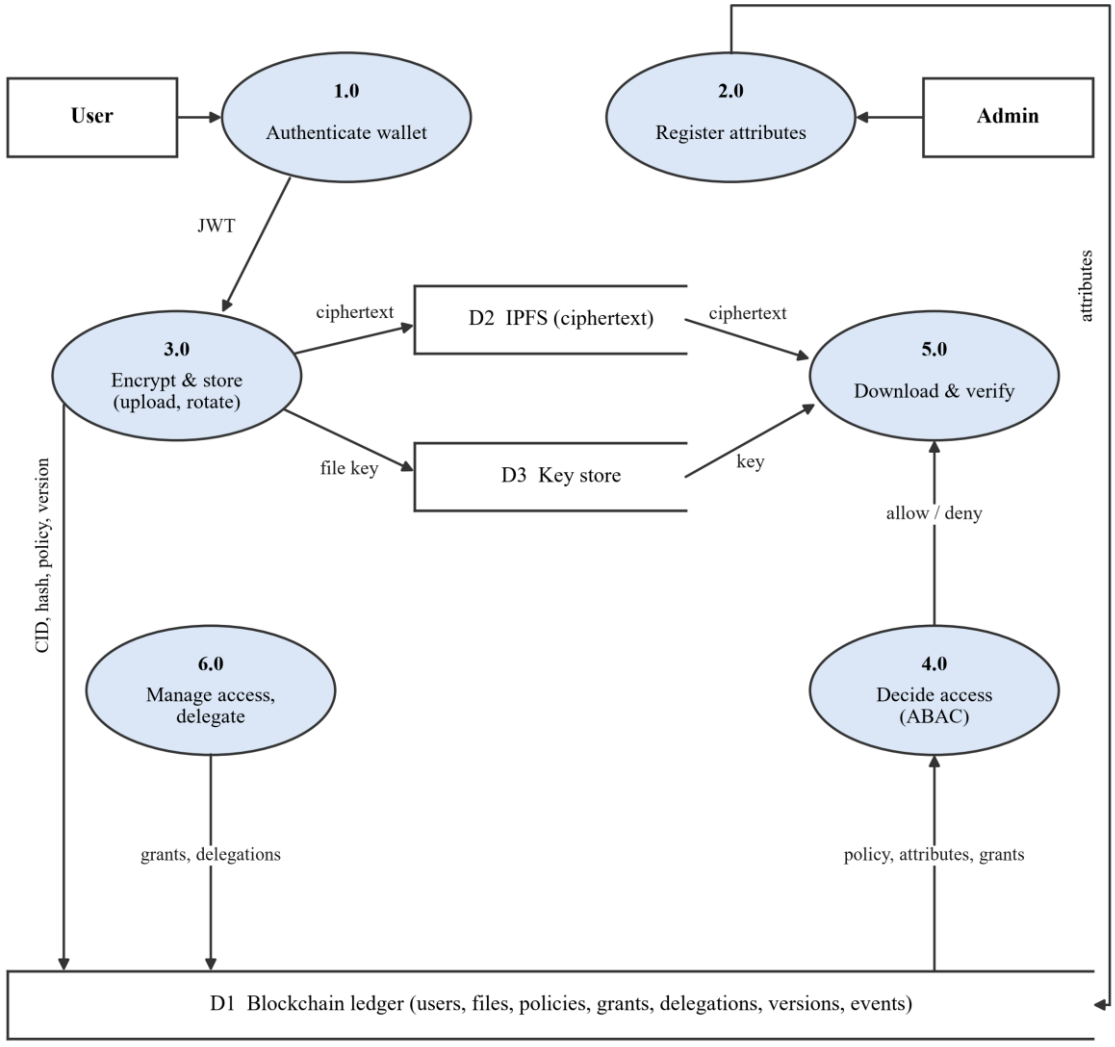


Figure 5.3: Data flow diagram – Level 1

5.2.1 Wallet Login Sequence

Figure 5.4 shows the challenge–response login. The backend issues a message containing a random nonce; the user signs it with MetaMask; the backend recovers the signer's address from the signature and, if it matches, issues a JWT. The nonce is deleted after one use, which prevents replay of an old signature.

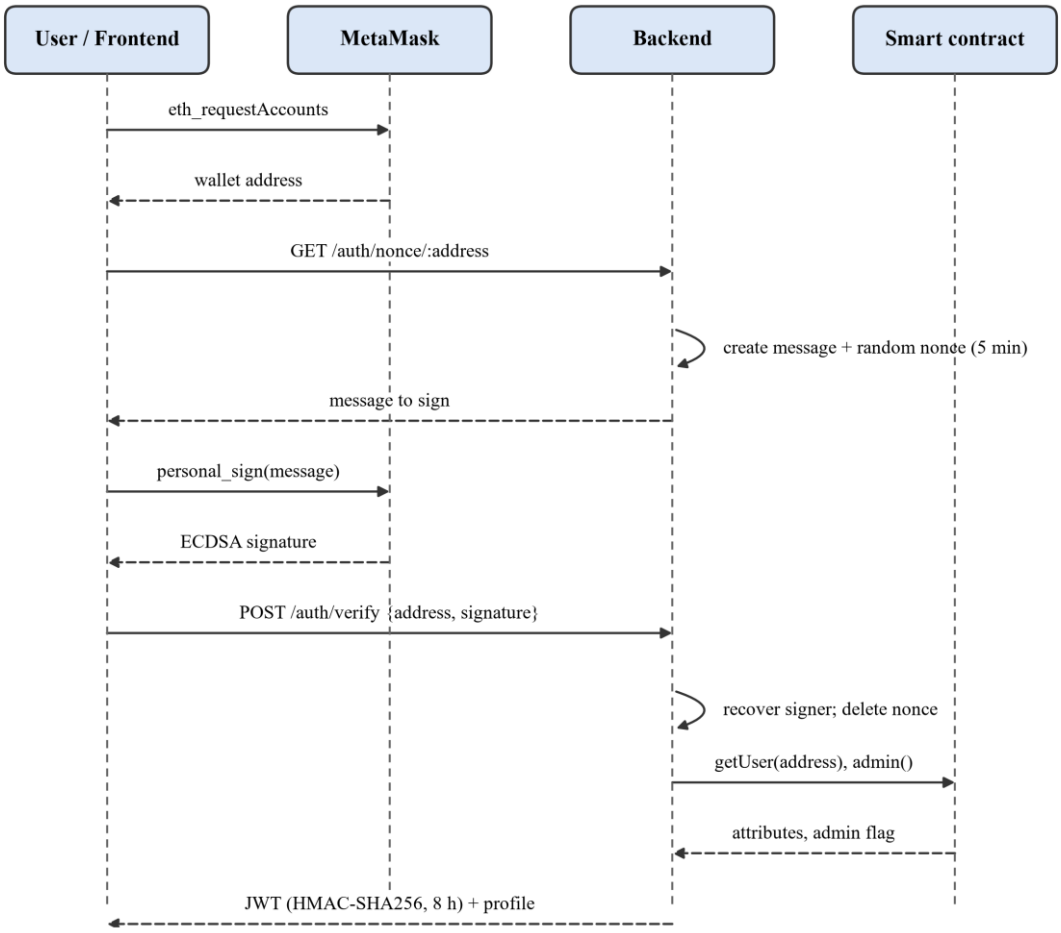


Figure 5.4: Sequence diagram – wallet login

5.2.2 File Upload Sequence

Figure 5.5 shows that encryption happens before the file reaches IPFS, and that the on-chain record is created by the user's own transaction, so the owner field in the contract is the user's verified address.

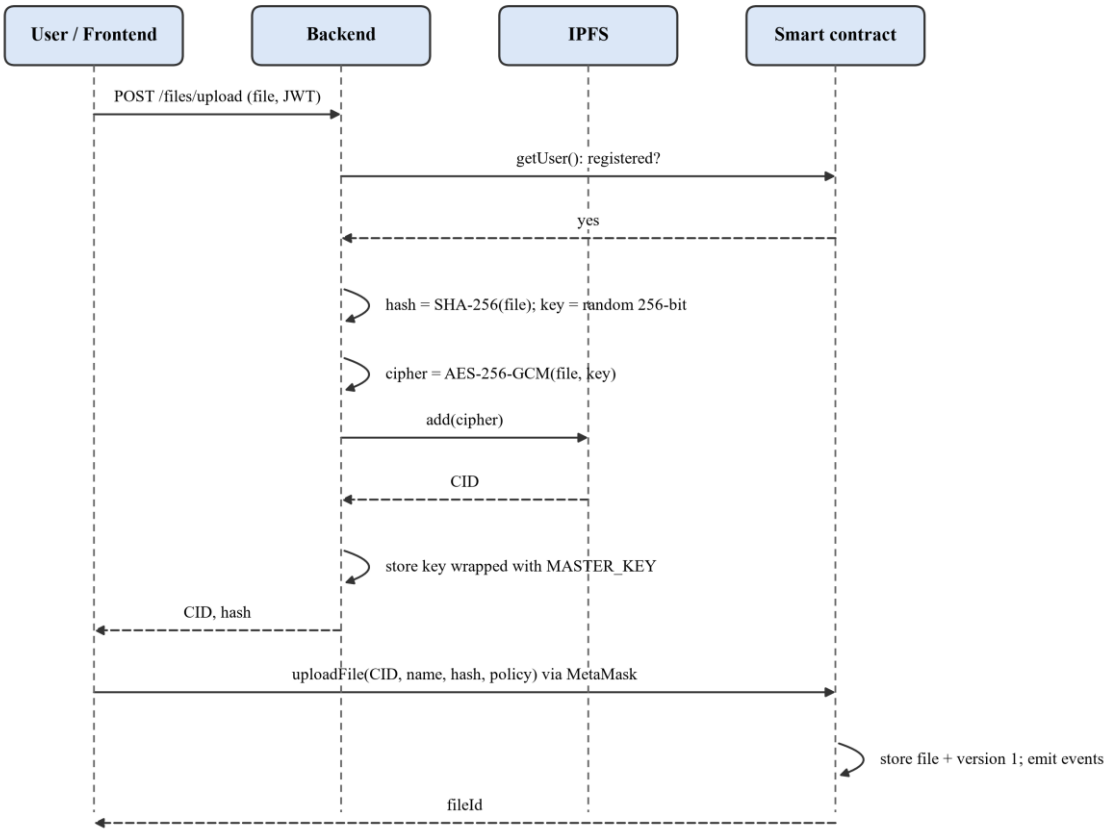


Figure 5.5: Sequence diagram – encrypted upload

5.3 Algorithms

The cryptographic building blocks are standard: AES-256-GCM (NIST SP 800-38D) for file encryption, SHA-256 (FIPS 180-4) for content hashes, ECDSA over secp256k1 with Keccak-256 hashing for wallet signatures (EIP-191 message format), HMAC-SHA256 for JWT sessions (RFC 7519), and a cryptographically secure random generator for keys, IVs and nonces. AES-256 encryption before IPFS storage is also used in recent systems [5], [6], and AES-256-GCM has been shown to secure IPFS data with minimal performance cost [9]. The algorithms below describe how the project combines them.

5.3.1 Attribute-Based Access Decision

The decision (Figure 5.6 and Algorithm 1) is implemented in the contract function `canAccess`. Rules are checked in a fixed order and the first rule that applies decides.

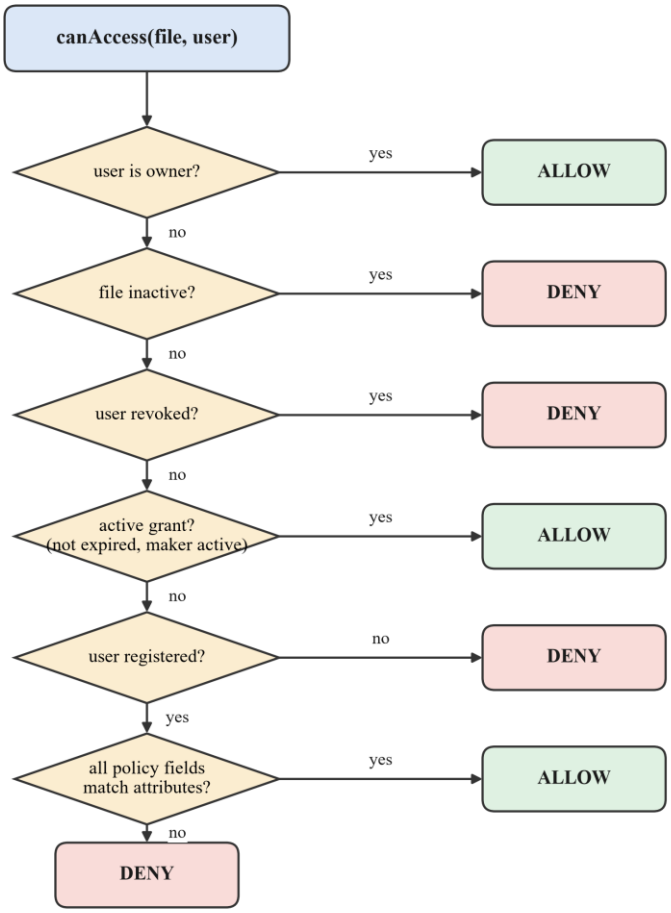


Figure 5.6: Flowchart of the on-chain access decision

Algorithm 1: canAccess(file f, user u)

Input: file record f, user address u
Output: true (allow) or false (deny)

1: if u = f.owner then return true
2: if f.active = false then return false
3: if revoked[f][u] then return false
4: if grantExpiry[f][u] > now and (grantedBy[f][u] = f.owner or isDelegate(f, grantedBy[f][u])) then return true
5: if users[u].registered = false then return false
6: for each field x in {role, department, organization}:
  7: if f.policy.x ≠ "" and f.policy.x ≠ users[u].x then return false
8: return true

5.3.2 Wallet Authentication

Algorithm 2: Challenge–response wallet login

Input: wallet address a
Output: JWT or rejection

1: server: n ← random 128-bit nonce; m ← "Sign in…" ‖ a ‖ n ‖ time; store (a, m) for 5 minutes
2: wallet: s ← ECDSA_sign(privateKey_a, m) (EIP-191 personal message)
3: server: remove stored (a, m); reject if missing or expired
4: server: a′ ← ECDSA_recover(m, s)
5: if a′ ≠ a then reject
6: return JWT = HMAC-SHA256(secret, {address: a, expiry: 8 h})

5.3.3 Encrypted Upload and Verified Download

Algorithm 3: Upload and download

Input: file F, policy P (upload); file id f, user u (download)
Output: file record (upload); F or denial (download)

1: upload: h ← SHA-256(F); k ← random 256-bit key; iv ← random 96-bit IV
2: upload: C ← iv ‖ tag ‖ AES-256-GCM_k(F); cid ← IPFS.add(C)
3: upload: store AES-GCM_masterKey(k) under cid
4: upload: user calls uploadFile(cid, name, h, P) → version 1
5: download: if canAccess(f, u) = false then return ACCESS DENIED
6: download: C ← IPFS.cat(f.cid); k ← unwrap(key[f.cid])
7: download: F′ ← AES-256-GCM decrypt (fails if C was modified)
8: download: if SHA-256(F′) ≠ f.contentHash then reject as tampered
9: download: return F′

5.3.4 Time-Limited Delegation (Novelty 1)

A delegation is a pair (file, delegate) with an end time stored in the contract. Figure 5.7 illustrates the rule that a delegate's grants never outlive its delegation.

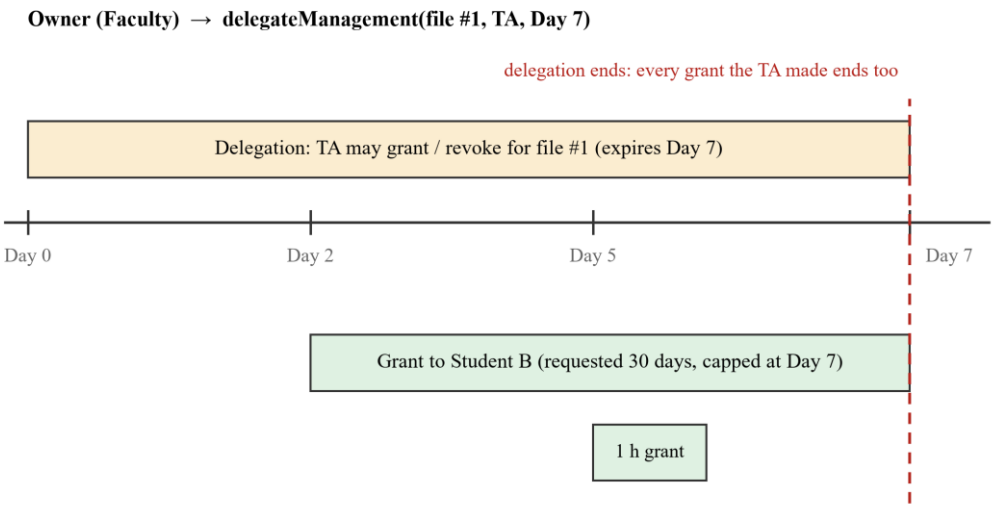


Figure 5.7: Lifetime of a delegation and the grants made under it

Algorithm 4: Delegated grant

Input: file f, caller c, target user t, requested end e
Output: grant recorded or rejected

1: require c = f.owner or delegationExpiry[f][c] > now
2: if c ≠ f.owner then
  3: require t ≠ c (no self-grant)
  4: require not (revoked[f][t] and revokedBy[f][t] = f.owner)
  5: e ← min(e, delegationExpiry[f][c])
6: revoked[f][t] ← false; grantExpiry[f][t] ← e; grantedBy[f][t] ← c
7: emit AccessGranted(f, t, c, e)

Because `canAccess` re-checks that the grant's maker is still the owner or an active delegate, removing a delegate early (`revokeDelegation`) immediately invalidates every grant it made, without touching each grant individually.

5.3.5 Revocation-Aware Key Rotation (Novelty 2)

Rotation is split into three phases (Figure 5.8) so that a rejected or failed transaction never leaves a file unreadable: the old key is destroyed only after the blockchain confirms the new version.

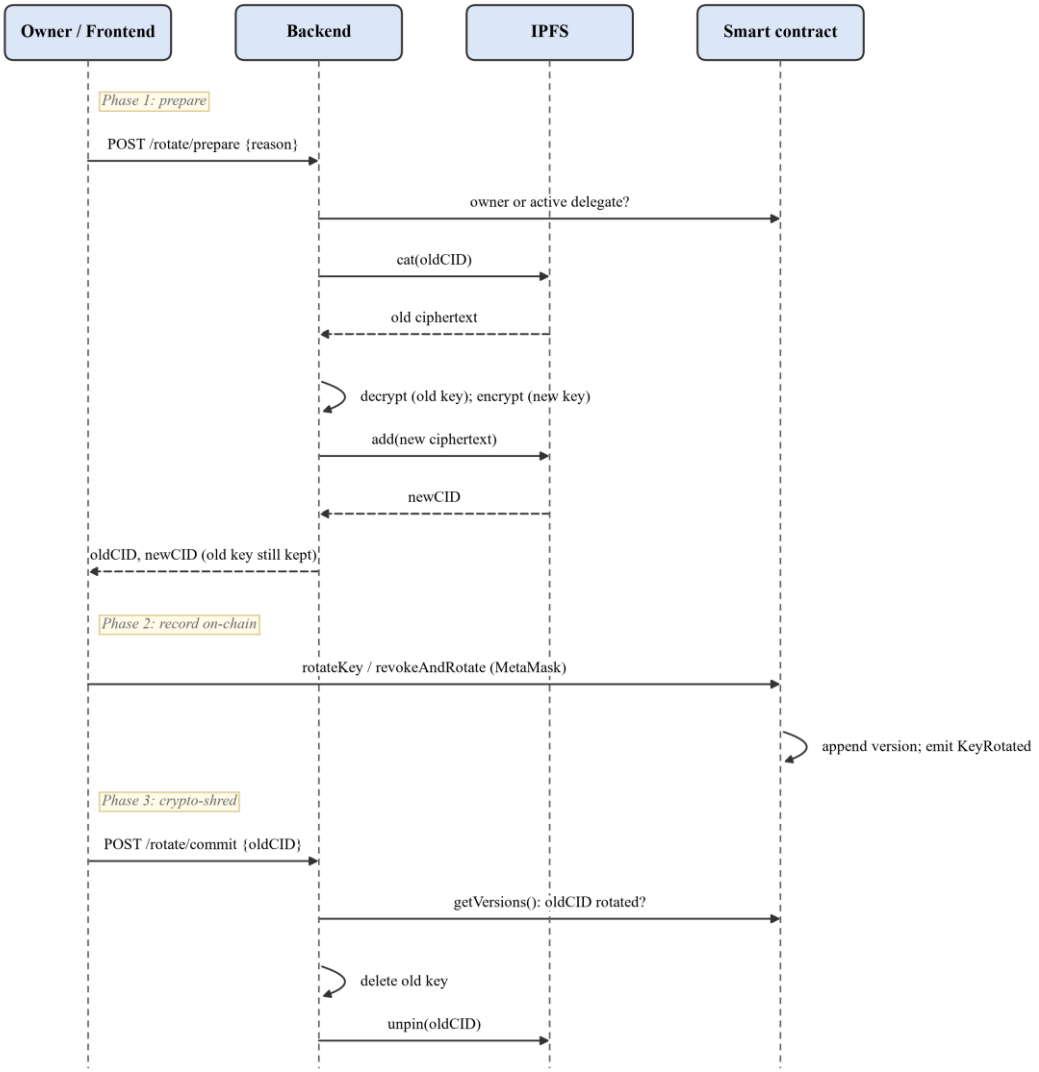


Figure 5.8: Sequence diagram – three-phase key rotation

Algorithm 5: Key rotation with crypto-shredding

Input: file f, reason r ∈ {Revocation, KeyCompromise}, optional user t to revoke
Output: new current version; old key destroyed

1: prepare: require caller is owner, or active delegate with r = Revocation
2: prepare: F ← decrypt(IPFS.cat(f.cid), key[f.cid])
3: prepare: k′ ← random 256-bit key; cid′ ← IPFS.add(AES-256-GCM_k′(F)); store k′
4: chain: if t given then revokeAndRotate(f, t, cid′) else rotateKey(f, cid′, r)
5: chain: append Version(cid′, f.contentHash, r, caller, now); f.cid ← cid′; emit KeyRotated
6: commit: require versions show oldCid followed by a rotation
7: commit: delete key[oldCid]; IPFS.unpin(oldCid)

5.4 Data Design

The project does not use a relational database; all trusted data lives in smart-contract storage. Figure 5.9 shows the on-chain data model as an entity–relationship view, and Table 5.1 maps each entity to its Solidity storage.

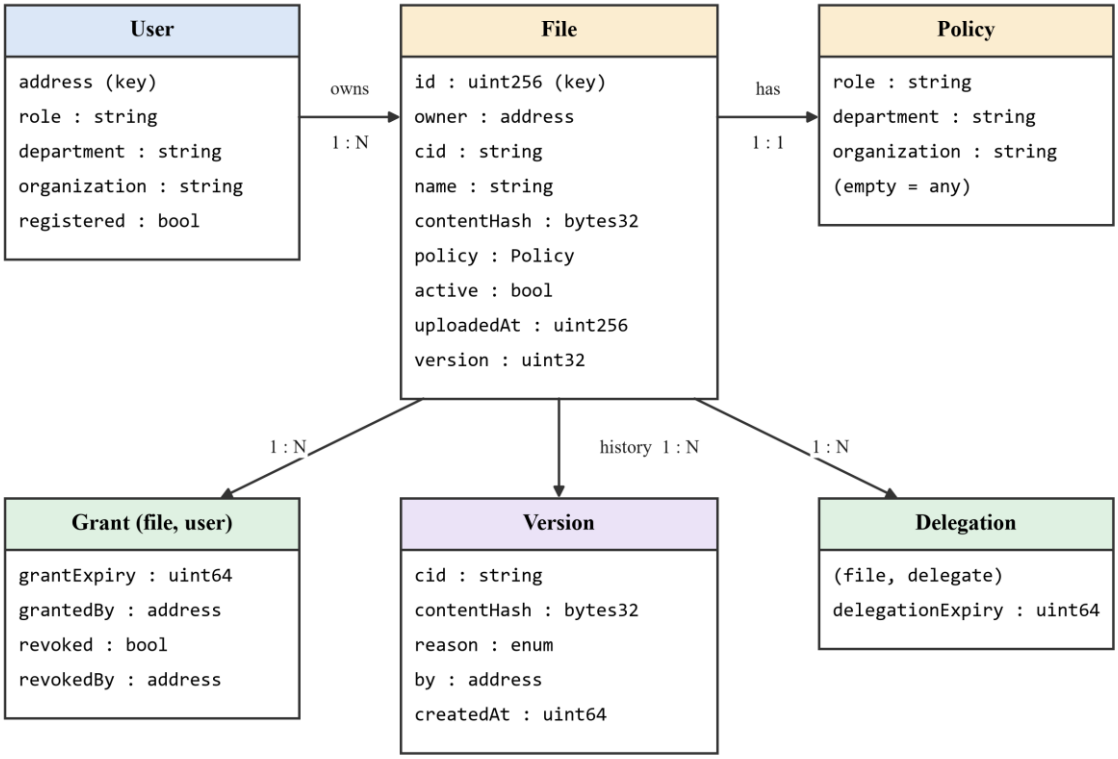


Figure 5.9: On-chain data model (entity–relationship view)

Table 5.1: Mapping of entities to contract storage

Entity	Solidity storage	Key
User	mapping(address => User) users	wallet address
File (+ Policy)	mapping(uint256 => File) files	file id (1, 2, …)
Version	mapping(uint256 => Version[]) fileVersions	file id, index
Grant / revocation	grantExpiry, grantedBy, revoked, revokedBy: mapping(uint256 => mapping(address => …))	file id, user
Delegation	delegationExpiry: mapping(uint256 => mapping(address => uint64)); delegate lists	file id, delegate
Audit record	Events: UserRegistered, FileUploaded, PolicyUpdated, AccessGranted, AccessRevoked, AccessRequested, FileStatusChanged, ManagementDelegated, DelegationRevoked, KeyRotated, VersionAdded	block, transaction

Off-chain, the backend keeps two stores: IPFS (or the local fallback directory) holding only ciphertext, and a key store (`keys.json`) mapping each CID to its AES key, itself encrypted with AES-256-GCM under the server's `MASTER_KEY`.

Chapter 6: Implementation Details

The implementation consists of about 4,200 lines of code across three packages in one repository: `blockchain/` (smart contract, tests, deployment), `backend/` (REST API) and `frontend/` (web application). A root script (`npm start`) starts a local blockchain, deploys the contract and runs the backend and frontend together.

6.1 Programming Languages Used

- **Solidity 0.8.24** for the `FileStorage` smart contract (454 lines). Solidity 0.8 checks integer overflow automatically; the contract is compiled with the optimizer and the via-IR pipeline.
- **JavaScript (Node.js 18)** for the backend services, controllers and routes, the deployment script, the start-up script and the automated tests.
- **JSX (React)** for the user-interface components and pages.
- **CSS** for styling, with light and dark colour schemes.

6.2 Technologies and Tools Used

Table 6.1: Technologies and their role in the project

Technology	Role in the project
Ethereum (local Hardhat network)	Executes the smart contract; chain ID 31337 with 20 funded test accounts; instant block for each transaction.
Hardhat, Mocha/Chai	Compiling Solidity, running 44 unit tests, time-travel helpers for testing expiry, deployment.
ethers.js v6	Reading contract state and verifying signatures (backend); sending MetaMask transactions (frontend).
MetaMask	Key custody, EIP-191 message signing for login, transaction confirmation.
IPFS (Kubo HTTP API)	Content-addressed storage of ciphertext (`/api/v0/add`, `/cat`, `/pin/rm`); automatic local fallback store.
Node.js crypto	AES-256-GCM, SHA-256 and secure random numbers.
Express, multer, jsonwebtoken	REST API, in-memory file upload (50 MB limit), JWT sessions.
React + Vite	Single-page web application and fast development server.
Git and GitHub	Version control; repository anshu3006/ipfs_blockchain.

6.2.1 Smart Contract Interface

Table 6.2: Main FileStorage contract functions

Function	Who may call	Purpose
registerUser	Admin	Set a wallet's role, department, organization
uploadFile	Registered user	Record CID, name, hash, policy; create version 1
updatePolicy / setFileActive	Owner	Change policy; deactivate or reactivate
grantAccess / grantTemporaryAccess	Owner, delegate	Explicit access, permanent or until a time
revokeAccess	Owner, delegate	Block a user even if attributes match
delegateManagement / revokeDelegation	Owner	Start or end a time-limited delegation
rotateKey / revokeAndRotate	Owner, delegate*	Record a re-encrypted version
addVersion	Owner	Record a new content version
requestAccess	Any user	Log an access attempt with its decision
canAccess, getFile, getVersions, getDelegates, …	Anyone (view)	Free read-only queries

The core of the access decision is a short view function. Because it runs on-chain, any party can verify exactly how a decision was made:

```
function canAccess(uint256 fileId, address user)
    public view fileExists(fileId) returns (bool)
{
    File storage f = files[fileId];
    if (user == f.owner) return true;
    if (!f.active) return false;
    if (revoked[fileId][user]) return false;
    if (hasActiveGrant(fileId, user)) return true;
    User storage u = users[user];
    if (!u.registered) return false;
    return _matches(f.policy.role, u.role)
        && _matches(f.policy.department, u.department)
        && _matches(f.policy.organization, u.organization);
}
```

6.2.2 Backend API

Table 6.3: Backend REST endpoints

Endpoint	Description
GET /api/auth/nonce/:address, POST /api/auth/verify	Wallet login: issue challenge, verify signature, return JWT
POST /api/files/upload	Encrypt file, add to IPFS, store wrapped key; return CID and hash

Endpoint	Description
GET /api/files/mine | shared | managed | :id	Own files, files shared with the user, files delegated to the user, one file
GET /api/files/:id/download	Check canAccess, decrypt, verify hash, return file
GET /api/files/:id/delegates	Active delegates of a file
GET /api/files/:id/versions, …/versions/:n/download	Version history; owner download of an older version
POST /api/files/:id/versions	Upload new content for the next version
POST /api/files/:id/rotate/prepare | commit	Two backend phases of key rotation
GET /api/access/:id/check, /audit, /user/:address	Decision with reason, audit log, user attributes

The frontend runs the three rotation phases as one action, so a rejected MetaMask transaction simply leaves the old version current:

```
export async function rotateKey(fileId, { reason, revokeUser = null }) {
  // 1. backend re-encrypts under a new key (old key still kept)
  const { oldCid, newCid } = await api.prepareRotation(fileId, reason);
  // 2. record the new version on-chain through MetaMask
  if (revokeUser) await chain.revokeAndRotate(fileId, revokeUser, newCid);
  else await chain.rotateKey(fileId, newCid, reason);
  // 3. destroy the old key and unpin the old CID
  return api.commitRotation(fileId, oldCid);
}
```

6.2.3 User Interface

The web application has seven pages: **Login** (connect wallet, sign, verify), **Dashboard** (identity, attributes, blockchain and IPFS status), **Upload** (file, policy and a two-step progress indicator), **My Files** (download and a Manage panel for policy, grants, revocation with optional key rotation, delegates, versions and file status), **Shared With Me** (files the user may open and request-by-ID), **Delegated to Me** (files the user manages as a delegate, with a limited panel), **Audit Log** (all contract events) and **Admin** (user registry, admin only).

6.3 Dataset Details

The project does not use a training or benchmark dataset. All data is generated during use: user attributes entered by the administrator and files uploaded by users. For testing, the automated tests create users with known attributes (for example Faculty/CSE/SIT and Student/ECE/SIT) and small text files, and the performance experiments in Chapter 7 use randomly generated files of 1 KB to 25 MB. The local network's public test accounts are used as wallets; they hold only test ether with no real value.

Chapter 7: Results

This chapter presents the working application, the results of automated and end-to-end testing, and measurements of gas cost and processing time. All measurements were taken on the development laptop (Intel Core i5-1235U, 8 GB RAM, Windows 11, Node.js 18.20.8) with the local Hardhat network and the local IPFS fallback store.

7.1 Screenshots

Figures 7.1 to 7.5 show the application running in Chrome with MetaMask connected to the local network.

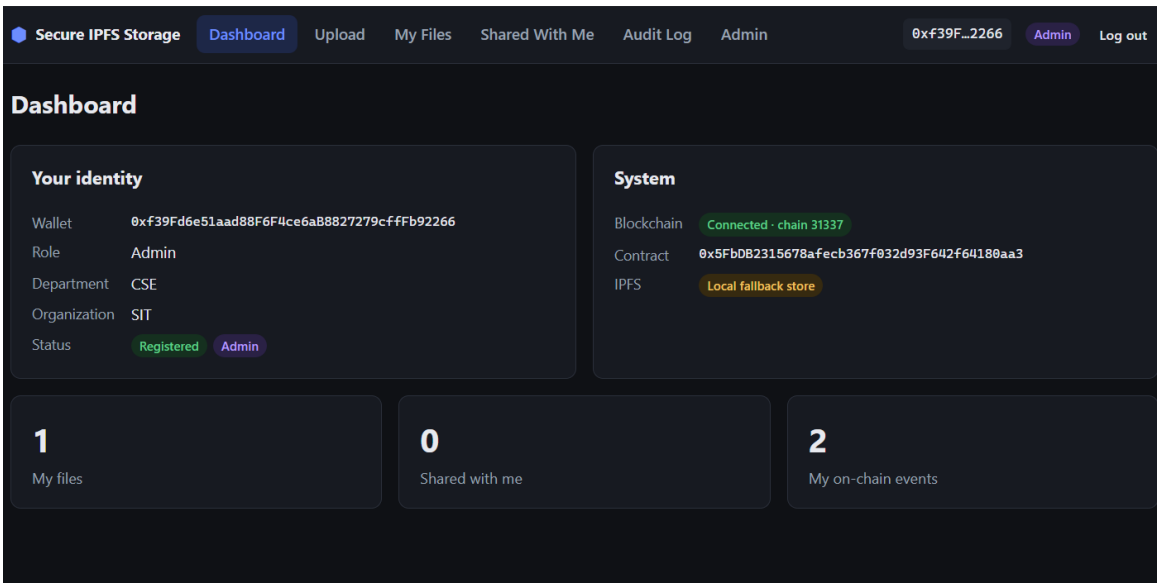


Figure 7.1: Dashboard of the administrator, with identity and system status

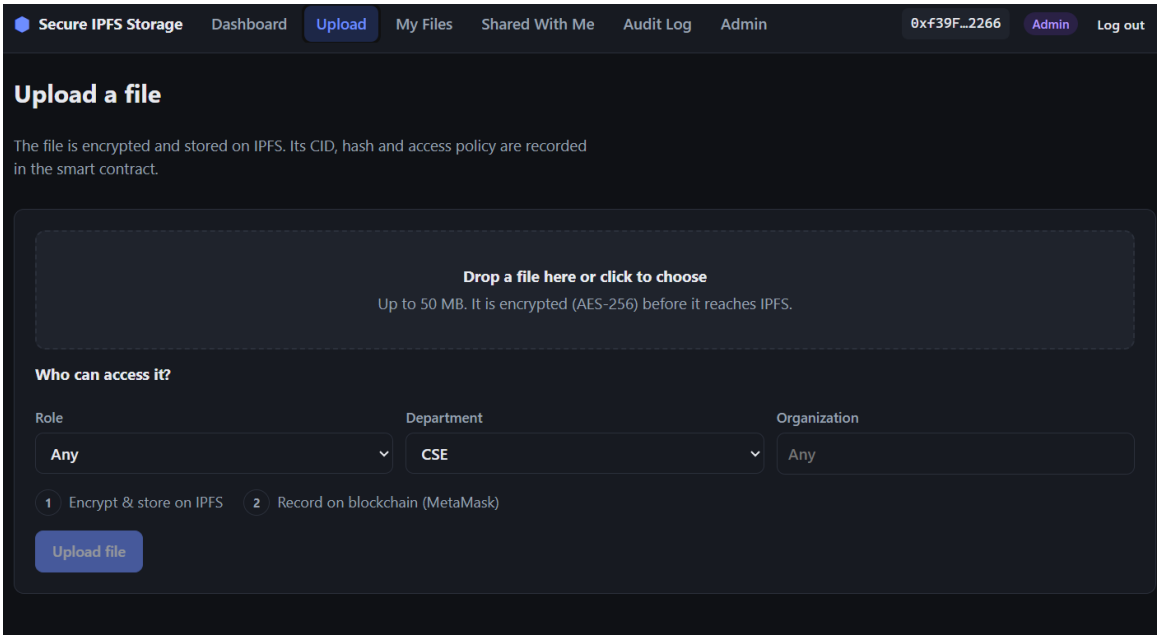


Figure 7.2: Upload page with the access policy and two-step progress

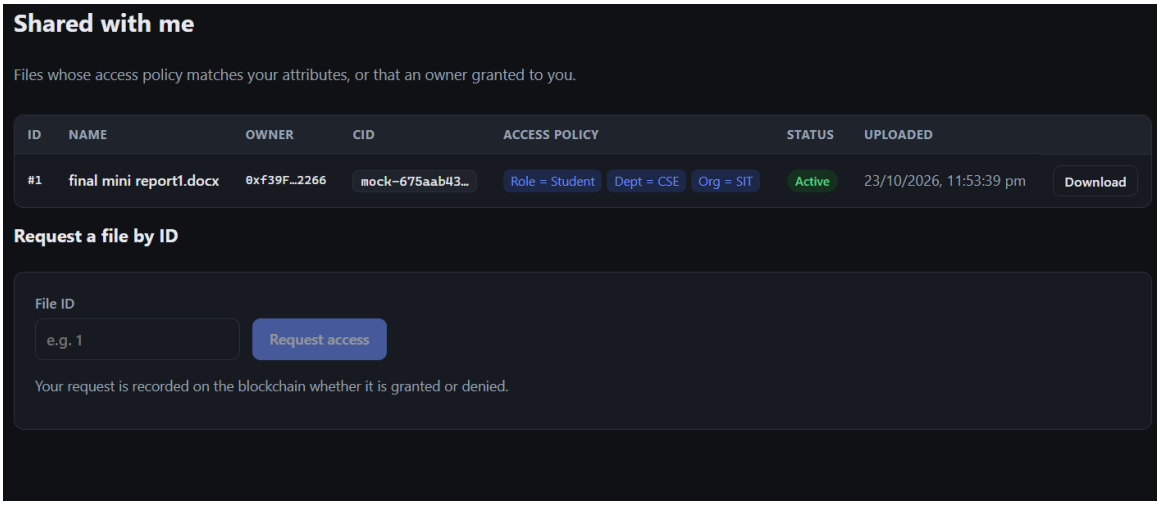


Figure 7.3: Shared With Me page for a CSE student, with request by ID

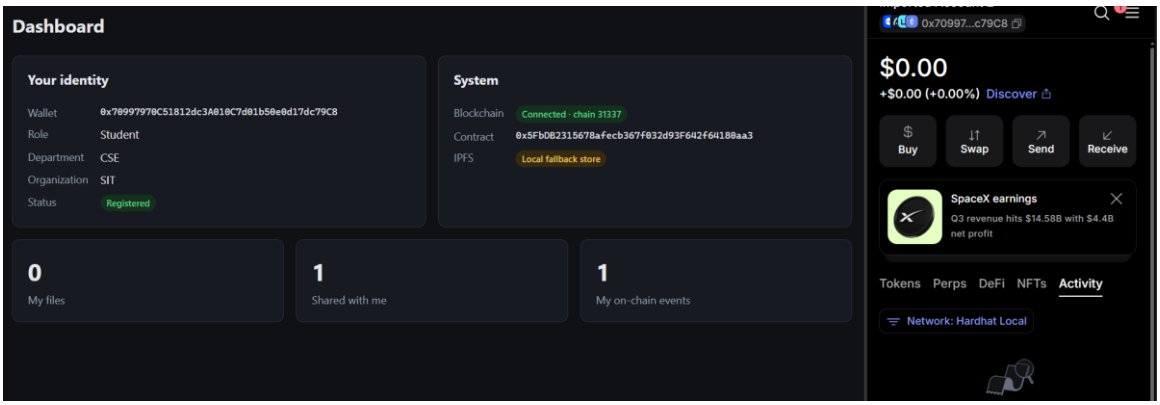


Figure 7.4: Dashboard of a registered student with the MetaMask wallet open

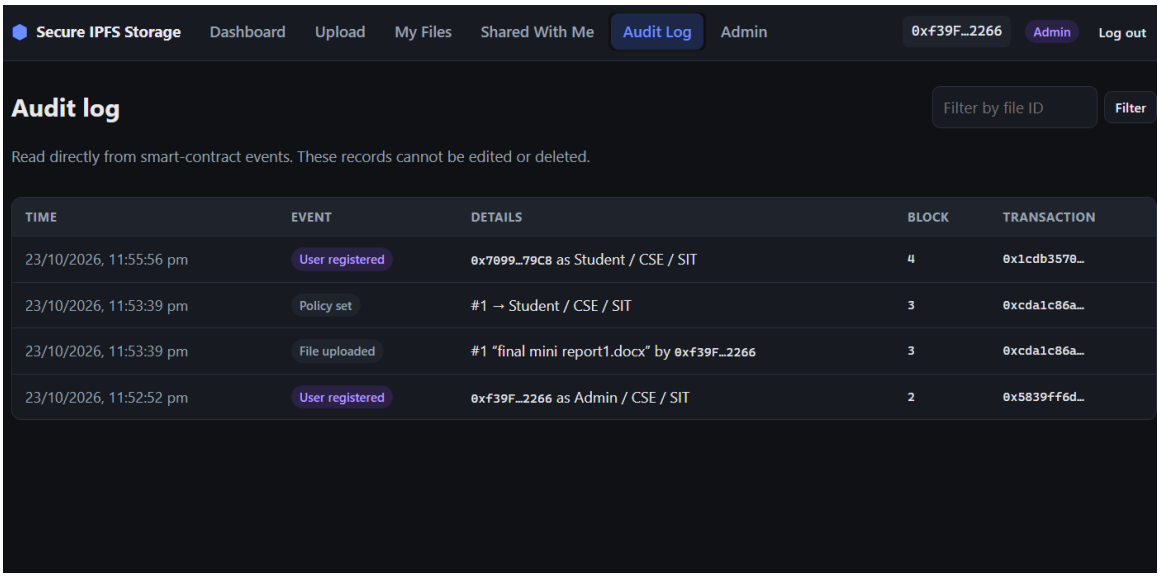


Figure 7.5: Audit log read directly from smart-contract events

7.2 Functional Testing

The smart contract is covered by 44 automated unit tests written with Hardhat, Mocha and Chai. Tests that involve time use Hardhat's time helpers to move the blockchain clock past an expiry. All 44 tests pass.

Table 7.1: Automated smart-contract tests

Test group	Tests	Examples of what is verified	Result
User registration	3	Deployer is admin; attributes stored; only admin can register	Pass
File upload	3	Record and event created; unregistered user and empty CID rejected	Pass
Attribute-based access	6	CSE→CSE allowed; ECE→CSE denied; all fields must match; owner always allowed	Pass
Grant, revoke, policy	7	Revocation overrides policy; explicit grant; policy change; deactivation; only owner manages	Pass
Temporary grants	2	Grant expires automatically; past expiry rejected	Pass
Delegation with limits	12	Delegate can grant/revoke; grants capped and cancelled with delegation; no self-grant; cannot undo owner's revoke or change policy	Pass
Versioning and key rotation	10	Rotation keeps hash and changes CID; revokeAndRotate is atomic; delegate only after revoke; only owner adds content versions	Pass
Audit log	1	Granted and denied requests are both recorded	Pass
Total	44		44 / 44

End-to-end tests were run against the complete stack (local blockchain, backend and contract) with scripted wallets. Table 7.2 lists the main scenarios; all passed, including all 19 checks of the key rotation and versioning flow.

Table 7.2: End-to-end test scenarios

Scenario	Expected result	Observed
Login with a valid wallet signature	JWT issued, profile returned	As expected
Upload as Faculty/CSE with policy Department = CSE	File encrypted, CID returned, record on-chain	As expected
Download as Student/CSE	Original content, hash verified	As expected
Download as Student/ECE	403 ACCESS DENIED, reason: department mismatch	As expected
Owner revokes the CSE student	Next download denied	As expected
Owner delegates to a TA; TA grants an ECE student for 1 h	ECE student allowed; reason shows delegate grant	As expected

Scenario	Expected result	Observed
TA tries to grant itself access	Rejected by contract	As expected
Owner removes the delegate	ECE student denied again	As expected
Revoke + rotate key	New CID, same hash, old key deleted and old copy removed	As expected
Commit before the chain records the rotation	409, old key kept	As expected
Download a shredded old version	410 Gone (key destroyed)	As expected
Upload a new content version	Users receive new content; owner can still download the previous version	As expected

7.3 Analysis

7.3.1 Gas Cost of Contract Operations

Gas measures the computational and storage work of a transaction on Ethereum. Table 7.3 lists the gas used by each operation, measured with the Hardhat network. Deploying the contract costs 3,354,186 gas once. Access decisions (`canAccess`) and all queries are view calls and cost no gas.

Table 7.3: Gas used by each contract transaction

Operation	Gas used	Purpose
registerUser	121,251	Store a user's three attributes
uploadFile	507,348	Create file record, policy and version 1
updatePolicy	61,139	Change the access policy
grantAccess	78,168	Permanent explicit grant
grantTemporaryAccess *	78,209	Grant with an end time
revokeAccess	73,815	Block a user
requestAccess	53,621	Log an access attempt
delegateManagement *	164,953	Start a delegation (first time)
revokeDelegation *	28,686	End a delegation early
rotateKey *	169,808	Record a re-keyed version
revokeAndRotate *	214,437	Revoke and re-key in one transaction
addVersion *	169,497	Record a new content version
setFileActive	27,809	Deactivate or reactivate a file

* Operations added by the two novel features.

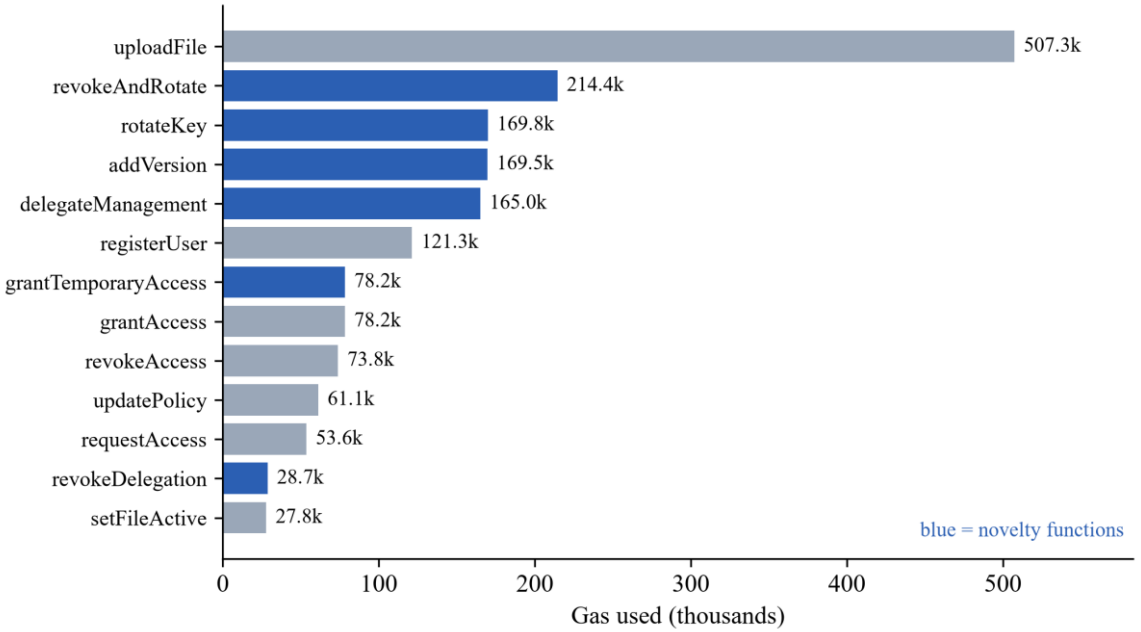


Figure 7.6: Gas used per operation (novelty functions in blue)

`uploadFile` is the most expensive operation because it stores several strings (CID, name, policy) and the first version entry. Day-to-day management operations such as granting, revoking and updating a policy cost 54k–78k gas. The novelty operations cost 29k–214k gas: a key rotation (about 170k gas) is cheaper than a fresh upload because only the new CID and a version entry are written, and `revokeAndRotate` saves a separate transaction by combining both steps.

7.3.2 Cryptographic Processing Time

Table 7.4 and Figure 7.7 show the time taken by the project's own encryption service for different file sizes (median of five runs). Encryption adds a fixed overhead of only 28 bytes per file (12-byte IV and 16-byte authentication tag).

Table 7.4: Encryption, decryption and hashing time

File size	AES-256-GCM encrypt (ms)	AES-256-GCM decrypt (ms)	SHA-256 (ms)
1 KB	0.05	0.03	0.01
100 KB	0.21	0.14	0.09
1 MB	1.41	1.05	0.86
10 MB	13.38	10.18	8.40
25 MB	50.99	37.91	24.42

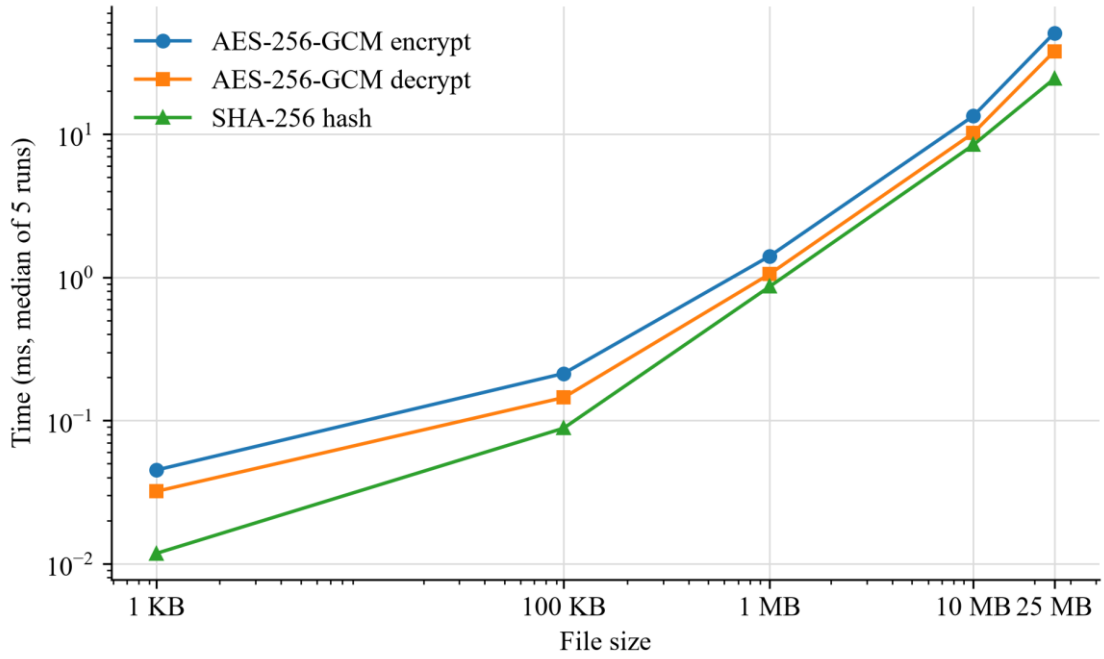


Figure 7.7: Cryptographic processing time versus file size (log–log)

Processing time grows linearly with file size, at roughly 1.3 ms per MB for encryption and 1 ms per MB for decryption. Even a 25 MB file is encrypted in about 51 ms, so encryption is not a bottleneck, which agrees with the minimal security–performance trade-off reported for AES-256-GCM on IPFS in [9].

7.3.3 End-to-End Latency

Table 7.5 and Figure 7.8 show complete operations through the running backend and local blockchain (median of three runs): encrypting and storing a file, the on-chain `uploadFile` transaction, the access-checked download, and a full three-phase key rotation.

Table 7.5: End-to-end operation time

File size	Encrypt + store (ms)	uploadFile tx (ms)	Check + download (ms)	Key rotation (ms)
1 KB	47.2	144.2	65.2	272.0
100 KB	35.4	144.5	65.4	233.7
1 MB	37.5	144.2	55.2	202.7
10 MB	106.8	149.3	130.8	291.4
25 MB	316.6	148.8	274.1	539.1

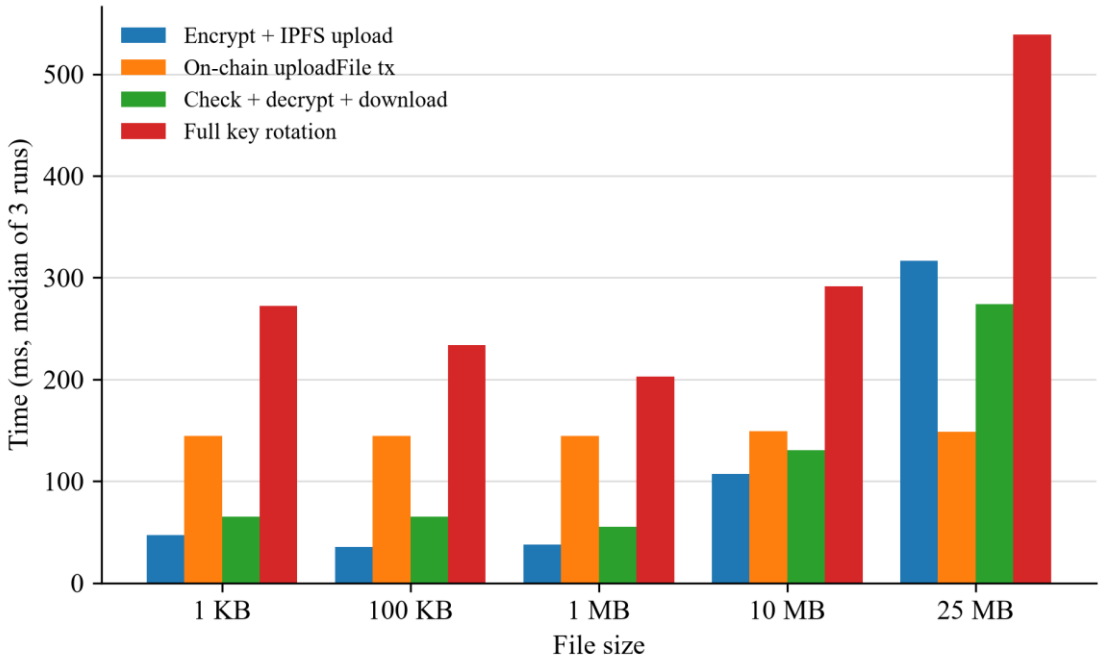


Figure 7.8: End-to-end latency by file size

A complete wallet login (challenge, MetaMask-style signature and verification) took 890 ms on the first request, including server warm-up. For files up to 1 MB every operation completes in under 0.3 s, dominated by fixed costs (HTTP and the blockchain transaction, about 145 ms) rather than file size. The on-chain transaction time is independent of file size, because only the CID and hash go on-chain. A full key rotation of a 25 MB file, including re-encryption, re-upload, the on-chain record and shredding the old key, takes about 0.54 s. On a public network, transaction time would instead depend on block time (about 12 s on Ethereum).

7.3.4 Comparison with Existing Work

Table 7.6: Feature comparison with the surveyed works

Work	IPFS	Encr.	ABAC	Audit	Revoke	Deleg.	Re-key	Versions	Open
[1] 2023	✓	✓	–	–	–	–	–	–	–
[2] 2026	–	✓	–	✓	✓	–	–	–	–
[3] 2026	✓	✓	✓	✓	✓	–	–	–	–
[4] 2026	✓	✓	–	✓	–	–	–	–	–
[5] 2025	✓	✓	–	✓	–	–	–	–	✓
[6] 2026	✓	✓	–	✓	✓	–	–	–	–
[7] 2025	✓	✓	✓	–	–	–	–	–	–
[8] 2026	–	✓	✓	✓	✓	–	✓*	–	–
[9] 2026	✓	✓	–	–	–	–	–	–	–
This work	✓	✓	✓	✓	✓	✓	✓	✓	✓

Columns: IPFS = files stored on IPFS; Encr. = data encrypted before storage; ABAC = attribute-based access control; Audit = on-chain audit of access activity; Revoke = revocation of individual users; Deleg. = time-limited delegation of management; Re-key = key rotation with destruction of old keys; Versions = on-chain file version history; Open = runs on a standard (non-permissioned) Ethereum network without special hardware. ✓ = provided; – = not described as a feature in the work as summarised in Chapter 2; ✓* = ciphertexts are re-encrypted on revocation, but old keys are not destroyed. [1] Antony Saviour and Samiappan; [2] Rajakarthik et al.; [3] Ahmed et al.; [4] Shaheen et al.; [5] Ghanmi et al.; [6] Hidayah and Ibrahim; [7] Li et al.; [8] Worapaluk and Fugkeaw; [9] El Sehayl et al.

Table 7.6 shows that this project is the only one of the ten systems that combines attribute-based access, user revocation, delegation, key rotation and version history. The closest works are [5], which uses the same Ethereum, MetaMask and IPFS stack but does not support user revocation or policy updates, and [6], which offers integrity checks, revocation and an audit trail with role-based access on a permissioned network.

The main trade-off compared with the CP-ABE designs of [3], [7] and [8] is that our backend holds the (wrapped) file keys and is trusted to release content only after the on-chain check. In return, the system needs no attribute-key generation, proxies or trusted hardware, applies policy changes and revocations instantly, and adds delegation, explained decisions, key rotation and versioning, all verified by tests. Unlike the permissioned deployments of [2], [4], [6] and [7],

it runs on a standard Ethereum network, so the same contract can be deployed to a public test network without a consortium set-up. Our measured on-chain transaction time (about 145 ms on a local network) is not directly comparable with the 4.5 s reported by [6] on a Besu network, since block production differs between networks.

Chapter 8: Conclusion

This project designed and implemented an immutable, distributed data-sharing framework that integrates IPFS storage with blockchain-driven authentication and attribute-based access control. Files are encrypted with AES-256-GCM before they reach IPFS; their CIDs, SHA-256 hashes and access policies are recorded in an Ethereum smart contract; users authenticate with wallet signatures instead of passwords; and access is decided transparently on-chain from users' role, department and organization. Every action is preserved as a tamper-proof audit event.

All five objectives from the synopsis were met, and two further objectives were achieved as the novelty of the work: time-limited, least-privilege **delegation of access management**, and **revocation-aware key rotation** with crypto-shredding and on-chain **file versioning**. The system was verified with 44 automated contract tests and end-to-end scenarios, and measurements showed that encryption is not a bottleneck, the on-chain cost is independent of file size, and common operations complete in well under a second on a local network.

8.1 Limitations

- The backend holds the wrapped file keys and is trusted to release a file only after the on-chain check; a fully trustless design would need attribute-based encryption.
- A copy that a user has already downloaded cannot be recalled by revocation or key rotation.
- User attributes are assigned by a single administrator rather than a decentralised identity provider.
- The demonstration runs on a local, in-memory Hardhat network and the local IPFS fallback store, so data does not persist after a restart.

8.2 Future Work

- Deploy the contract to a public test network such as Sepolia and run a real IPFS Kubo node for persistent, publicly verifiable storage.
- Support richer policies with AND/OR conditions and additional attributes such as year of study, with attribute expiry.
- Add an access-request and approval workflow and notifications for grants, revocations and delegations.

- Explore threshold or split-key storage so that no single server can decrypt files, and watermarking to trace leaked downloads.
- Integrate decentralised identity (verifiable credentials) for attributes instead of an admin registry.

References

[1] M. Antony Saviour and D. Samiappan, "IPFS based file storage access control and authentication model for secure data transfer using block chain technique," Concurrency and Computation: Practice and Experience, vol. 35, no. 2, e7485, 2023, doi: 10.1002/cpe.7485.

[2] R. Rajakarthik, V. Sumalatha and R. Rajalingam, "Decentralized Privacy Preserving Data Sharing Model for Electronic Health Records (EHR) Using Blockchain Technology," IET Blockchain, vol. 6, e70037, 2026, doi: 10.1049/blc2.70037.

[3] F. Ahmed, T. Zhou, H. Bilal, N. Kumar, A. M. Almuhaideb and S. Ayouni, "Enhancing Railway Network Security and Privacy With Blockchain and Attribute-Based Encryption," IEEE Transactions on Intelligent Transportation Systems, early access, 2026, doi: 10.1109/TITS.2026.3725650.

[4] R. Shaheen, A. S. Khan, I. A. Abbasi and Y. Gulzar, "Blockchain-IPFS Integrated Permissioned Architecture for Secure, Scalable and Encrypted Healthcare Data Governance," Journal of Cyber Security and Risk Auditing, vol. 2026, no. 3, 2026.

[5] H. Ghanmi, N. Hajlaoui, H. Touati, S. Boudjit, M. Hadded and P. Muhlethaler, "A blockchain-based framework for secure data sharing and access control using IPFS and ECC," Cluster Computing, vol. 28, art. 1007, 2025, doi: 10.1007/s10586-025-05703-4.

[6] S. S. Hidayah and I. Ibrahim, "Design and Development of a File Sharing System with Integrity Assurance and Audit Trail using Permissioned Blockchain and IPFS: A Case Study of DISPUSIPDA West Java," Sistemasi: Jurnal Sistem Informasi, vol. 15, no. 5, pp. 1900–1917, 2026.

[7] Q. Li, W. Liao, Z. Liang, X. Zhang, M. Xu and G. Jin, "Research on Trusted Storage and Dynamic Access Control of Smart Grid Data Based on Blockchain and IPFS," in Proc. 2025 IEEE 7th International Conference on Communications, Information System and Computer Engineering (CISCE), 2025, doi: 10.1109/CISCE65916.2025.11065401.

[8] K. Worapaluk and S. Fugkeaw, "Blockchain-Enabled Privacy-Preserving Access Control for EHRs Sharing With Optimized User and Attribute Revocation," IET Information Security, vol. 2026, art. 3917525, 2026, doi: 10.1049/ise2/3917525.

[9] M. A. El Sehayl, A. Almaaz, K. Mershad and N. Abbas, "Optimizing and securing the IPFS protocol using a parallelized AES-256 GCM engine," Journal of Cyber Security Technology, 2026, doi: 10.1080/23742917.2026.2625461.